2. CORE SFRS

MIKROELEKTRONIKA

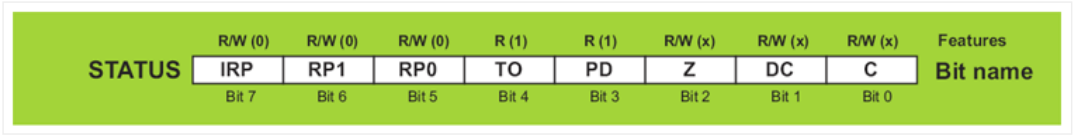
Features and Function

The special function registers can be classified into two categories:

- Core (CPU) registers – control and monitor operation and processes in the central processor. Even though there are only a few of them, the operation of the whole microcontroller depends on their contents.
- Peripheral SFRs- control the operation of peripheral units (serial communication module, A/D converter etc.). Each of these registers is mainly specialized for one circuit and for that reason they will be described along with the circuit they are in control of.

The core (CPU) registers of the PIC16F887 microcontroller are described in this chapter. Since their bits control several different circuits within the chip, it is not possible to classify them into some special group. These bits are described along with the processes they control.

STATUS Register



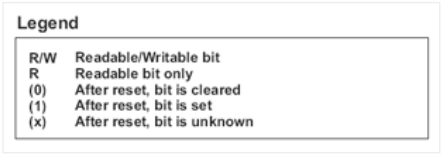


Fig. 2-1 STATUS Register

The STATUS register contains: the arithmetic status of the W register, the RESET status and the bank select bits for data memory. One should be careful when writing a value to this register because if you do it wrong, the results may be different than expected. For example, if you try to clear all bits using the `CLRF STATUS` instruction, the result in the register will be 000xx1xx instead of the expected 00000000. Such errors occur because some of the bits of this register are set or cleared according to the hardware as well as because the bits 3 and 4 are readable only. For these reasons, if it is required to change its content (for example, to change active bank), it is recommended to use only instructions which do not affect any Status bits (C, DC and Z). Refer to “Instruction Set Summary”.

- **IRP** – Bit selects register bank. It is used for indirect addressing.
  - **1** – Banks 0 and 1 are active (memory location 00h-FFh)
  - **0** – Banks 2 and 3 are active (memory location 100h-1FFh)
- **RP1,RP0** – Bits select register bank. They are used for direct addressing.

RP1	RP0	ACTIVE BANK

0	0	Bank0
0	1	Bank1
1	0	Bank2
1	1	Bank3

Table 2-1

- **TO – Time-out bit.**
  - **1** – After power-on or after executing `CLRWDT` instruction which resets watch-dog timer or SLEEP instruction which sets the microcontroller into low-consumption mode.
  - **0** – After watch-dog timer time-out has occurred.
- **PD – Power-down bit.**
  - **1** – After power-on or after executing `CLRWDT` instruction which resets watch-dog timer.
  - **0** – After executing `SLEEP` instruction which sets the microcontroller into low-consumption mode.
- **Z – Zero bit**
  - **1** – The result of an arithmetic or logic operation is zero.
  - **0** – The result of an arithmetic or logic operation is different from zero.
- **DC – Digit carry/borrow** bit is changed during addition and subtraction if an “overflow” or a “borrow” of the result occurs.
  - **1** – A carry-out from the 4th low-order bit of the result has occurred.
  - **0** – No carry-out from the 4th low-order bit of the result has occurred.
- **C – Carry/Borrow bit** is changed during addition and subtraction if an “overflow” or a “borrow” of the result occurs, i.e. if the result is greater than 255 or less than 0.
  - **1** – A carry-out from the most significant bit of the result has occurred.
  - **0** – No carry-out from the most significant bit of the result has occurred.

OPTION_REG Register

Fig.2-2

The OPTION_REG register contains various control bits to configure: Timer0/WDT prescaler, timer TMR0, external interrupt and pull-ups on PORTB.

- **RBPU – Port B Pull up Enable bit.**
  - **1** – PortB pull-ups are disabled.
  - **0** – PortB pull-ups are enabled.

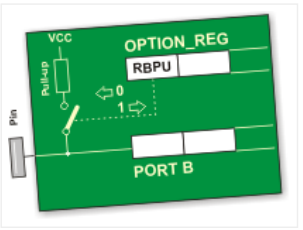


Fig.2-3

- **INTEDG – Interrupt Edge Select bit.**
  - **1** – Interrupt on rising edge of RB0/INT pin.
  - **0** – Interrupt on falling edge of RB0/INT pin.

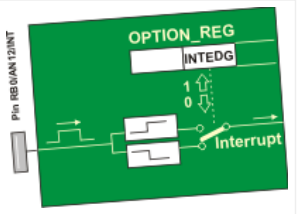


Fig.2-4

- **T0CS – TMR0 Clock Source Select bit.**
  - **1** – Transition on TOCKI pin.
  - **0** – Internal instruction cycle clock (Fosc/4).

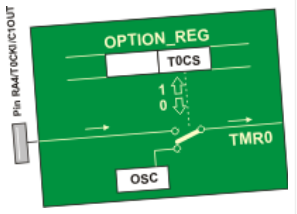


Fig.2-5

- **T0SE – TMR0 Source Edge Select bit** selects pulse edge (rising or falling) counted by the timer TMR0 through the RA4/T0CKI pin.
  - **1** – Increment on high-to-low transition on TOCKI pin.
  - **0** – Increment on low-to-high transition on TOCKI pin.

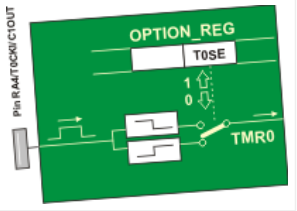


Fig.2-6

- **PSA – Prescaler Assignment bit** assigns prescaler (only one exists) to the timer or watchdog timer.
  - **1** – Prescaler is assigned to the WDT.
  - **0** – Prescaler is assigned to the TMR0.

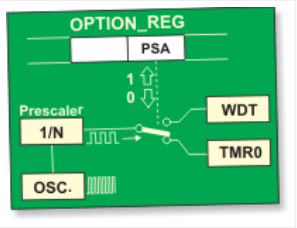


Fig.2-7

PS2, PS1, PS0 Prescaler Rate Select bits

Prescaler rate is selected by combining these three bits. Described, as shown in the table below, prescaler rate depends on whether prescaler is assigned (TMR0) or watch-dog timer (WDT).

PS2	PS1	PS0	TMR0	WDT
0	0	0	1:2	1:1
0	0	1	1:4	1:2
0	1	0	1:8	1:4
0	1	1	1:16	1:8

1	0	1	1:64	1:32
1	1	0	1:128	1:64
1	1	1	1:256	1:128

Table 2-2

In order to achieve 1:1 prescaler rate when the timer TMR0 counts up pulses, the prescaler should be assigned to the WDT. As a result of this, the timer TMR0 does not use the prescaler, but directly counts pulses generated by the oscillator, which was the objective!

Interrupt System Registers

When an interrupt request arrives it does not mean that interrupt will automatically occur, because it must also be enabled by the user (from within the program). Because of that, there are special bits used to enable or disable interrupts. It is easy to recognize these bits by IE contained in their names (stands for Interrupt Enable). Besides, each interrupt is associated with another bit called the flag which indicates that interrupt request has arrived regardless of whether it is enabled or not. They are also easily recognizable by the last two letters contained in their names- IF (Interrupt Flag).

As seen, everything is based on a simple and efficient idea. When an interrupt request arrives, the flag bit is to be set first.

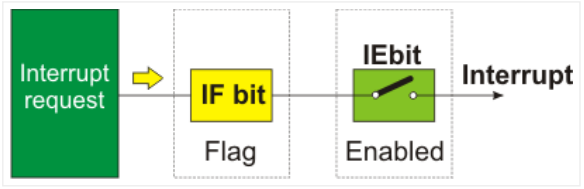


Fig. 2-8 Interrupt System Registers

If the appropriate IE bit is not set (0), this event will be completely ignored. Otherwise, an interrupt occurs! In case several interrupt sources are enabled, it is necessary to detect the active one before the interrupt routine starts execution. Source detection is performed by checking flag bits.

It is important to understand that the flag bits are not automatically cleared, but by software during interrupt routine execution. If this detail is neglected, another interrupt will occur immediately upon return to the program, even though there are no more requests for its execution! Simply put, the flag as well as IE bit remained set.

All interrupt sources typical of the PIC16F887 microcontroller are shown on the next page. Note several things:

- **GIE bit** – enables all unmasked interrupts and disables all interrupts simultaneously.
- **PEIE bit** – enables all unmasked peripheral interrupts and disables all peripheral interrupts (This does not concern Timer TMR0 and port B interrupt sources).

To enable interrupt caused by changing logic state on port B, it is necessary to enable it for each bit separately. In this case, bits of the **IOCB**register have the function to control IE bits.

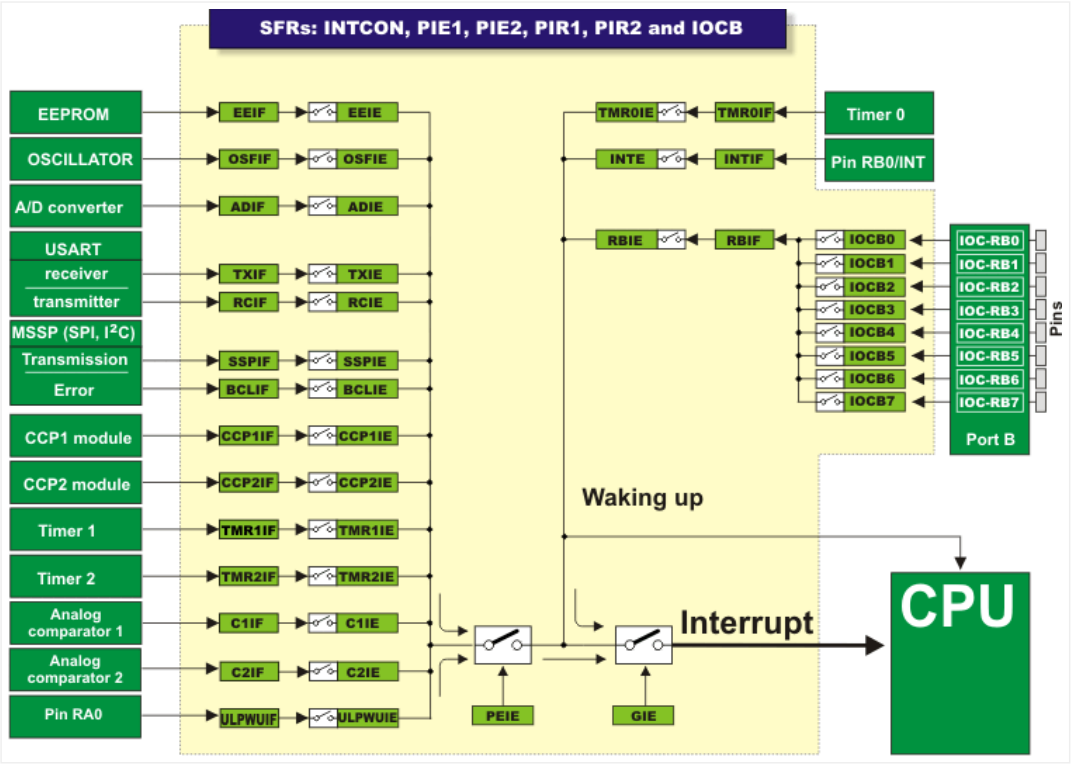


Fig. 2-9 Interrupt SFRs

INTCON Register

The INTCON register contains various enable and flag bits for TMR0 register overflow, PORTB change and external INT pin interrupts.

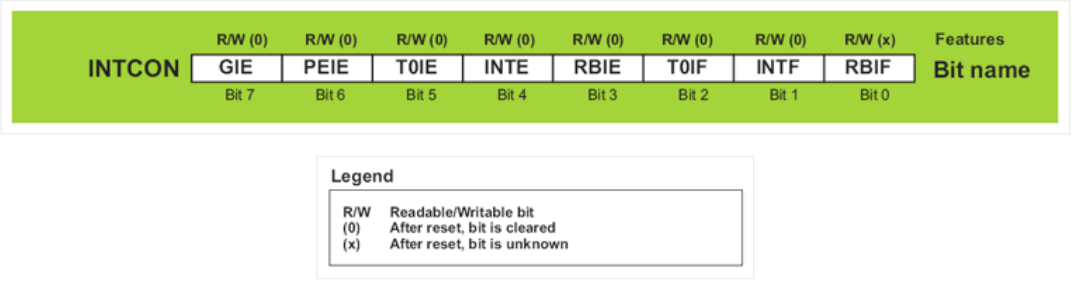


Fig. 2-10 INTCON Register

- **GIE – Global Interrupt Enable bit** – controls all possible interrupt sources simultaneously.
  - **1** – Enables all unmasked interrupts.
  - **0** – Disables all interrupts.
- **PEIE – Peripheral Interrupt Enable bit** acts similar to GIE, but controls interrupts enabled by peripherals. It means that it does not affect interrupts triggered by the timer TMR0 or by changing state on port B or RB0/INT pin.
  - **1** – Enables all unmasked peripheral interrupts.
  - **0** – Disables all peripheral interrupts.
- **T0IE – TMR0 Overflow Interrupt Enable bit** controls interrupt enabled by TMR0 overflow.
  - **1** – Enables the TMR0 interrupt.
  - **0** – Disables the TMR0 interrupt.
- **INTE – RB0/INT External Interrupt Enable bit** controls interrupt caused by changing logic state on pin RB0/IN (external interrupt).
  - **1** – Enables the INT external interrupt.
  - **0** – Disables the INT external interrupt.

- **RBIE – RB Port Change Interrupt Enable bit**. When configured as inputs, port B pins may cause interrupt by changing their logic state (no matter whether it is highto- low transition or vice versa, fact that something is changed only matters). This bit determines whether interrupt is to occur or not.
  - **1** – Enables the port B change interrupt.
  - **0** – Disables the port B change interrupt.
- **T0IF – TMR0 Overflow Interrupt Flag bit** registers the timer TMR0 register overflow, when counting starts from zero.
  - **1** – TMR0 register has overflowed (bit must be cleared in software).
  - **0** – TMR0 register has not overflowed.
- **INTF – RB0/INT External Interrupt Flag bit** registers change of logic state on the RB0/INT pin.
  - **1** – The INT external interrupt has occurred (must be cleared in software).
  - **0** – The INT external interrupt has not occurred.
- **RBIF – RB Port Change Interrupt Flag bit** registers change of logic state of some port B input pins.
  - **1** – At least one of the port B general purpose I/O pins has changed state. Upon reading portB, RBIF (flag bit) must be cleared in software.
  - **0** – None of the port B general purpose I/O pins has changed state.

PIE1 Register

The PIE1 register contains the peripheral interrupt enable bits.

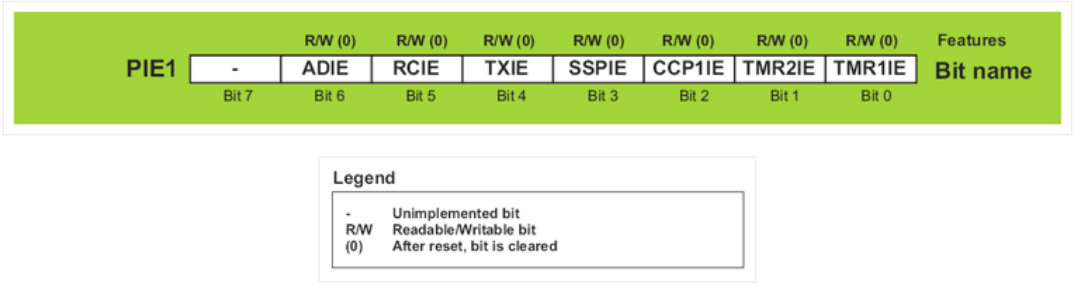


Fig. 2-11 PIE1 register

- **ADIE – A/D Converter Interrupt Enable bit.**
  - **1** – Enables the ADC interrupt.
  - **0** – Disables the ADC interrupt.
- **RCIE – EUSART Receive Interrupt Enable bit.**
  - **1** – Enables the EUSART receive interrupt.
  - **0** – Disables the EUSART receive interrupt.
- **TXIE – EUSART Transmit Interrupt Enable bit.**
  - **1** – Enables the EUSART transmit interrupt.
  - **0** – Disables the EUSART transmit interrupt.
- **SSPIE – Master Synchronous Serial Port (MSSP) Interrupt Enable bit** – enables an interrupt request to be generated after each data transfer via synchronous serial communication module (SPI or I2C mode).
  - **1** – Enables the MSSP interrupt.
  - **0** – Disables the MSSP interrupt.
- **CCP1IE – CCP1 Interrupt Enable bit** enables an interrupt request to be generated in CCP1 module used for PWM signal processing.
  - **1** – Enables the CCP1 interrupt.
  - **0** – Disables the CCP1 interrupt.
- **TMR2IE – TMR2 to PR2 Match Interrupt Enable bit**

  - **1** – Enables the TMR2 to PR2 match interrupt.
  - **0** – Disables the TMR2 to PR2 match interrupt.

- **TMR1IE – TMR1 Overflow Interrupt Enable bit** enables an interrupt request to be generated after each timer TMR1 register overflow, i.e. when the counting starts from zero.
  - **1** – Enables the TMR1 overflow interrupt.
  - **0** – Disables the TMR1 overflow interrupt.

PIE2 Register

The PIE2 Register also contains the various interrupt enable bits.

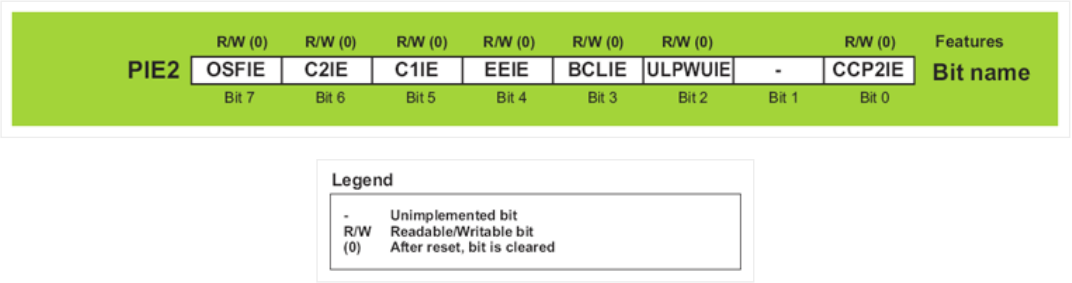


Fig. 2-12 PIE2 Register

- **OSFIE – Oscillator Fail Interrupt Enable bit.**
  - **1** – Enables oscillator fail interrupt.
  - **0** – Disables oscillator fail interrupt.

- **C2IE – Comparator C2 Interrupt Enable bit.**
  - **1** – Enables Comparator C2 interrupt.
  - **0** – Disables Comparator C2 interrupt.

- **C1IE – Comparator C1 Interrupt Enable bit.**
  - **1** – Enables Comparator C1 interrupt.
  - **0** – Disables Comparator C1 interrupt.

- **EEIE – EEPROM Write Operation Interrupt Enable bit.**
  - **1** – Enables EEPROM write operation interrupt.
  - **0** – Disables EEPROM write operation interrupt.

- **BCLIE – Bus Collision Interrupt Enable bit.**
  - **1** – Enables bus collision interrupt.
  - **0** – Disables bus collision interrupt.

- **ULPWUIE – Ultra Low-Power Wake-up Interrupt Enable bit.**
  - **1** – Enables Ultra Low-Power Wake-up interrupt.
  - **0** – Disables Ultra Low-Power Wake-up interrupt.

- **CCP2IE – CCP2 Interrupt Enable bit.**
  - **1** – Enables CCP2 interrupt.
  - **0** – Disables CCP2 interrupt.

PIR1 Register

The PIR1 register contains the interrupt flag bits.

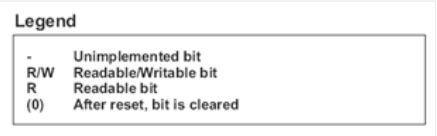


Fig. 2-13 PIR1 Register

- **ADIF – A/D Converter Interrupt Flag bit.**
  - **1** – A/D conversion is completed (bit must be cleared in software).
  - **0** – A/D conversion is not completed or has not started.
- **RCIF – EUSART Receive Interrupt Flag bit.**
  - **1** – The EUSART receive buffer is full. Bit is cleared by reading the RCREG register.
  - **0** – The EUSART receive buffer is not full.
- **TXIF – EUSART Transmit Interrupt Flag bit.**
  - **1** – The EUSART transmit buffer is empty. Bit is cleared by writing to the TXREG register.
  - **0** – The EUSART transmit buffer is full.
- **SSPIF – Master Synchronous Serial Port (MSSP) Interrupt Flag bit.**
  - **1** – The MSSP interrupt condition during data transmit/receive has occurred. These conditions differ depending on MSSP operating mode (SPI or I2C) This bit must be cleared in software before returning from the interrupt service routine.
  - **0** – No MSSP interrupt condition has occurred.
- **CCP1IF – CCP1 Interrupt Flag bit.**
  - **1** – CCP1 interrupt condition has occurred (CCP1 is unit for capturing, comparing and generating PWM signal). Depending on operating mode, capture or compare match has occurred. In both cases, bit must be cleared in software. This bit is not used in PWM mode.
  - **0** – No CCP1 interrupt condition has occurred.
- **TMR2IF – Timer2 to PR2 Interrupt Flag bit**
  - **1** – TMR2 (8-bit register) to PR2 match has occurred. This bit must be cleared in software before returning from the interrupt service routine.
  - **0** – No TMR2 to PR2 match has occurred.
- **TMR1IF – Timer1 Overflow Interrupt Flag bit**
  - **1** – The TMR1 register has overflowed. This bit must be cleared in software.
  - **0** – The TMR1 register has not overflowed.

PIR2 Register

The PIR2 register contains the interrupt flag bits.

Fig. 2-14 PIR2 register

- **OSFIF – Oscillator Fail Interrupt Flag bit.**
  - **1** – System oscillator failed and clock input has changed to internal oscillator INTOSC. This bit must be cleared in software.

  - **0** – System oscillator operates normally.

- **C2IF – Comparator C2 Interrupt Flag bit.**
  - **1** – Comparator C2 output has changed (bit C2OUT). This bit must be cleared in software.
  - **0** – Comparator C2 output has not changed.

- **C1IF – Comparator C1 Interrupt Flag bit.**
  - **1** – Comparator C1 output has changed (bit C1OUT). This bit must be cleared in software.
  - **0** – Comparator C1 output has not changed.

- **EEIF – EE Write Operation Interrupt Flag bit.**
  - **1** – EEPROM write completed. This bit must be cleared in software.
  - **0** – EEPROM write is not completed or has not started.

- **BCLIF – Bus Collision Interrupt Flag bit.**
  - **1** – A bus collision has occurred in the MSSP when configured for I2C Master mode. This bit must be cleared in software.
  - **0** – No bus collision has occurred.

- **ULPWUIF – Ultra Low-power Wake-up Interrupt Flag bit.**
  - **1** – Wake-up condition has occurred. This bit must be cleared in software.
  - **0** – No Wake-up condition has occurred.

- **CCP2IF – CCP2 Interrupt Flag bit.**
  - **1** – CCP2 interrupt condition has occurred (unit for capturing, comparing and generating PWM signal). Depending on operating mode, capture or compare match has occurred. In both cases, the bit must be cleared in software. This bit is not used in PWM mode.
  - **0** – No CCP2 interrupt condition has occurred.

PCON register

The PCON register contains only two flag bits used to differentiate between a: power-on reset, brown-out reset, Watchdog Timer Reset and external reset (through MCLR pin).

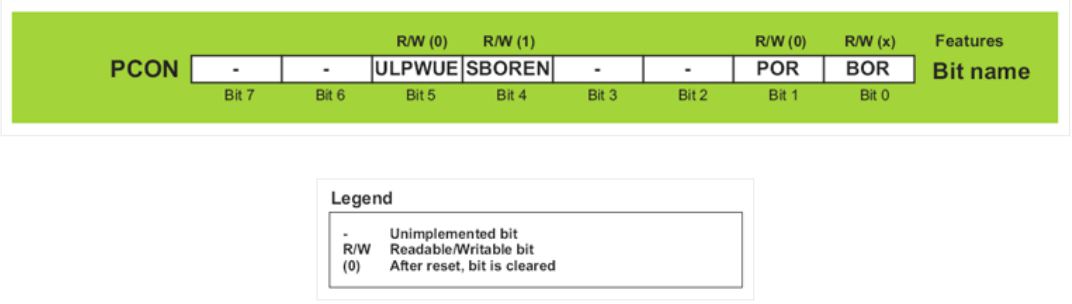


Fig. 2-15 PCON register

- **ULPWUE – Ultra Low-Power Wake-up Enable bit**
  - **1** – Ultra Low-Power Wake-up enabled.
  - **0** – Ultra Low-Power Wake-up disabled.

- **SBOREN – Software BOR Enable bit**
  - **1** – Brown-out Reset enabled.
  - **0** – Brown-out Reset disabled.

- **POR – Power-on Reset Status bit**
  - **1** – No Power-on reset has occurred.
  - **0** – Power-on reset has occurred. This bit must be set in software after a Power-on Reset occurs.

- **BOR – Brown-out Reset Status bit**
  - **1** – No Brown-out reset has occurred.

– **0** – Brown-out reset has occurred. This bit must be set in software after a Brown-out Reset occurs.

PCL and PCLATH Registers

The size of the program memory of the PIC16F887 is 8K. Therefore, it has 8192 locations for program storing. For this reason the program counter must be 13-bits wide (2^13 = 8192). In order that the contents of some location may be changed in software during operation, its address must be accessible through some SFR. Since all SFRs are 8-bits wide, this register is “artificially” created by dividing its 13 bits into two independent registers: PCLATH and PCL.

If the program execution does not affect the program counter, the value of this register is automatically and constantly incremented +1, +1, +1, +1… In that way, the program is executed just as it is written- instruction by instruction, followed by a constant address increment.

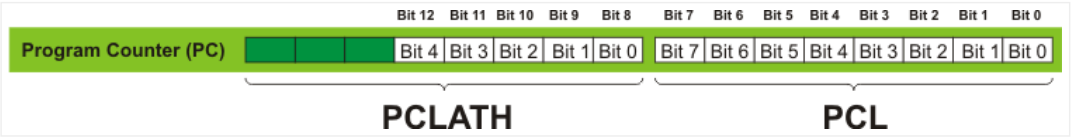


Fig. 2-16 PCL and PCLATH Registers

If the program counter is changed in software, then there are several things that should be kept in mind in order to avoid problems:

- Eight lower bits (the low byte) come from the PCL register which is readable and writable, whereas five upper bits coming from the PCLATH register are writable only.
- The PCLATH register is cleared on any reset.
- In assembly language, the value of the program counter is marked with PCL, but it obviously refers to 8 lower bits only. One should take care when using the “`ADDWF PCL`” instruction. This is a jump instruction which specifies the target location by adding some number to the current address. It is often used when jumping into a look-up table or program branch table to read them. A problem arises if the current address is such that addition causes change on some bit belonging to the higher byte of the PCLATH register. Do you see what is going on?
  Executing any instruction upon the PCL register simultaneously causes the Prog ram Counter bits to be replaced by the contents of the PCLATH register. However, the PCL register has access to only 8 lower bits of the instruction result and the following jump will be completely incorrect. The problem is solved by setting such instructions at addresses ending by xx00h. This enables the program to jump up to 255 locations. If longer jumps are executed by this instruction, the PCLATH register must be incremented by 1 for each PCL register overflow.
- On subroutine call or jump execution (instructions `CALL` and `GOTO`), the microcontroller is able to provide only 11-bit addressing. For this reason, similar to RAM which is divided in “banks”, ROM is divided in four “pages” in size of 2K each. Such instructions are executed within these pages without any problems. Simply, since the processor is provided with 11-bit address from the program, it is able to address any location within 2KB. Figure 2-17 below illustrates this situation as a jump to the subroutine PP1 address.
  However, if a subroutine or jump address are not within the same page as the location from where the jump is, two “missing”- higher bits should be provided by writing to the PCLATH register. It is illustrated in figure 2-17 below as a jump to the subroutine PP2 address.

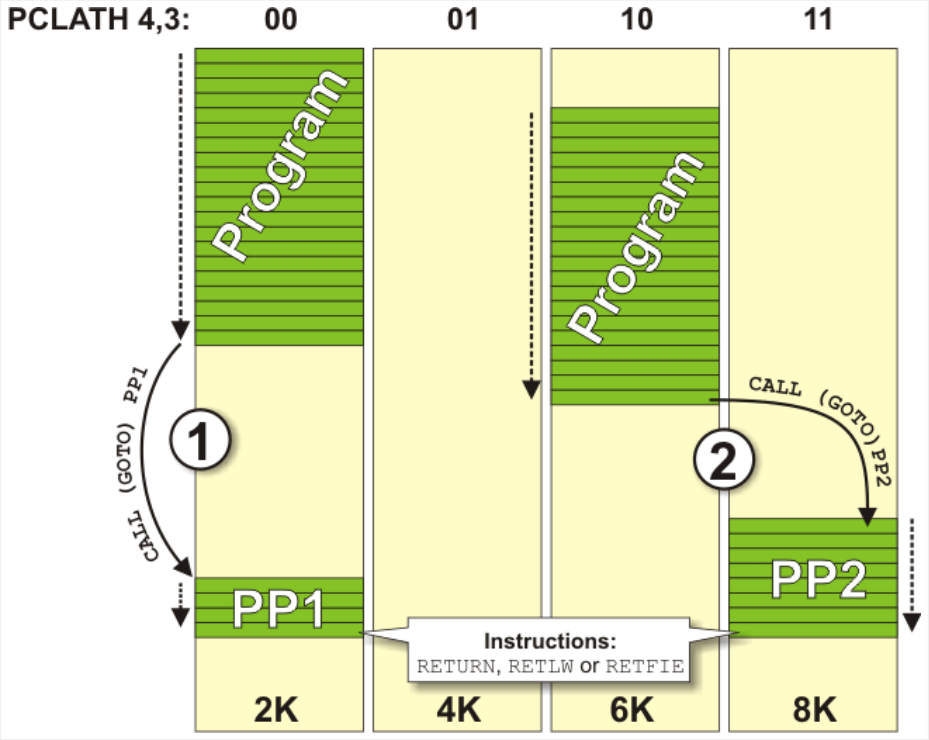


Fig. 2-17 PCLATH Registers

In both cases, when the subroutine reaches instructions `RETURN`, `RETLW` or `RETFIE` (to return to the main program), the microcontroller will simply continue program execution from where it left off because the return address is pushed and saved onto the stack which, as mentioned, consists of 13-bit registers.

Indirect addressing

In addition to direct addressing which is logical and clear by itself (it is sufficient to specify address of some register to read its contents), this microcontroller is able to perform indirect addressing by means of the INDF and FSR registers. It sometimes considerably simplifies program writing. The whole procedure is enabled because the INDF register is not true one (physically does not exist), but only specifies the register whose address is located in the FSR register. Because of this, write or read from the INDF register actually means write or read from the register whose address is located in the FSR register. In other words, registers' addresses are specified in the FSR register, and their contents are stored in the INDF register. The difference between direct and indirect addressing is illustrated in the figure 2-18 below:

As seen, the problem with the "missing addressing bits" is solved by "borrowing" from another register. This time, it is the seventh bit called IRP from the STATUS register.

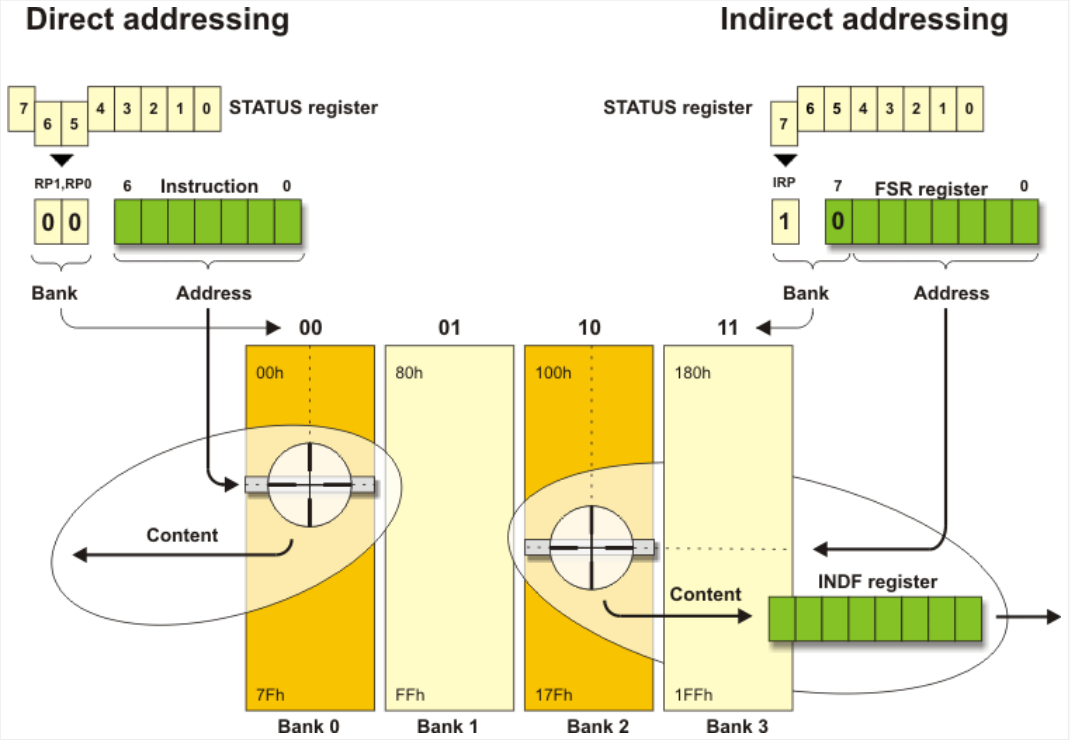


Fig. 2-18 Direct and Indirect addressing